

UNIOESTE – Universidade Estadual do Oeste do Paraná
Campus de Foz do Iguaçu
Ciência da Computação - CECE
Organização e Arquitetura de Computadores
Prof. Fabiana Frata Furlan Peres
Alunas: Julia Gabrielly Kao e Julia Pavan Doebber

Relatório do Técnico – Analisador Léxico

Este relatório apresenta o analisador léxico desenvolvido para a linguagem Pakao, baseada na linguagem C. Para a construção deste software, utilizou-se o Flex, uma ferramenta de código aberto para a geração de analisadores léxicos, e o Visual Studio Code (VS Code) como Ambiente de Desenvolvimento Integrado (IDE).

O desenvolvimento também fez uso de bibliotecas padrão de C, como <stdlib.h> para a leitura de arquivos, <stdio.h> para entrada e saída de dados, e <string.h> para a manipulação de cadeias de caracteres.

Funcionamento do software

O programa implementa a primeira fase do processo de compilação: a análise léxica. A partir da leitura de um arquivo txt informado pelo usuário, será analisado todos os caracteres e utilizá das expressões regulares criadas para identificar e agrupar esses caracteres lidos em tokens válidos.

Os tokens reconhecidos armazenados sequencialmente em uma lista encadeada e serão impressos no terminal no final da execução. Caso sejam encontrados erros ou caracteres que não pertençam à definição da linguagem, o sistema realiza o tratamento adequado.

Além da identificação de padrões, o analisador léxico prevê quatro tipos específicos de tratamento de erro. Ao encontrar entradas inválidas, o sistema reporta a falha detalhando o tipo do erro e a sua localização exata, apresentando essas informações em conjunto com os tokens reconhecidos no terminal.

Tokens definidos e tratamento de erros

Para a construção do analisador léxico, foram construídos 14 tokens, sendo eles:

Quadro 1 – Tokens definidos para o analisador léxico

Token	Expressão regular
DIGITO	[0-9]
LETRA	[a-zA-Z]
IDENTIFICADOR	{LETRA}{(LETRA){(DIGITO)}'_'}
RESERVADA	("while" "for" "if" "else" "do" "int" "void" "float" "double" "long" "than" "boo " "char" "string" "return" "break" "struct" "type" "static" "const" "def" "imp ort" "try" "catch" "throw")
INTEIRO	("+" "-"){DIGITO}+
FLOAT	{INTEIRO}*"."{DIGITO}+
STRING	\["^\\n"]*
CHAR	\{LETRA\}{DIGITO}\'
OPERADORARIT	("+" "-" "*" "/" "x" "." "*" "++" "--")
OPERADORREL	("==" "!=" ">" "<" ">=" "<=")
OPERADORLOG	("&&" " " "!")
OPERADORTRIB	("+=" "-= "*= " /= "=")"
SEPARADORES	("," "." ";" ":")

Além dos tokens, foram definidos 4 tratamentos de erros:

Quadro 2 – Tratamentos de erros definidos para o analisador léxico

Tipo de erro	Erro
Identificador mal formato	{DIGITO}+{(LETRA) "_" }+{(LETRA){(DIGITO)} "_" }
String nao fechada	\["^\\n"]*\n
Float sem casas decimais	{DIGITO}+ "."
Caractere invalido	.

Métodos implementados

Para a execução do analisador léxico foram utilizadas as seguintes funções:

- **adicionar_token(const char* classe, const char* lexema):** Função responsável por alocar dinamicamente a memória para um novo nó da Lista

Encadeada (TokenNode), inserindo a classe do token e o valor exato lido (yytext) no final da estrutura.

- **imprimir_lista():** Função que percorre toda a Lista Encadeada armazenada na memória, formatando e imprimindo no console a tabela de tokens na exata ordem de aparecimento no código-fonte.
- **yylex():** Função principal gerada pelo Flex. Ela é o autômato finito, que consome o arquivo de entrada caractere por caractere, casando as expressões regulares e executando as ações (como chamar o adicionar_token ou contabilizar colunas).
- **yywrap():** Função auxiliar exigida pelo Flex para sinalizar que a leitura do arquivo chegou ao fim (retorna 1).
- **main(int argc, char argv):** Ponto de entrada do programa. Recebe o nome do arquivo fonte passado via terminal, aponta a leitura do Flex (yyin) para este arquivo, chama o yylex() para executar a análise e, ao final, aciona imprimir_lista().

Execução do software

Para a execução do analisador léxico, é necessário ter o compilador GCC (via MinGW) e a ferramenta Flex previamente instalados no sistema operacional Windows. Atendidos esses pré-requisitos, o arquivo trab_1.l deve ser aberto no Visual Studio Code. Em seguida, os seguintes comandos devem ser executados no terminal da IDE

1. flex trab_1.l (Para gerar o código em C a partir do arquivo Flex)
2. GCC:gcc lex.yy.c -o analisador.exe (Para compilar o código gerado pelo passo anterior)

Com a compilação concluída, é possível testar o analisador passando um arquivo de texto como parâmetro. Para isso, digite o seguinte comando no terminal:

```
./analisador.exe nome.txt
```

É preciso colocar o nome do arquivo no lugar de “nome”.

Referências Bibliográficas

AHO, A.V; LAM, M.S.; SETHI, R.; ULLMAN, J.D. **Compiladores: princípios, técnicas e ferramentas**. Pearson. 2008.

DELAMARO, M.E. **Como construir um compilador utilizando Ferramentas Java**. Novatec Editora. 2004.

LOUDEN, K.C. **Compiladores: princípios e práticas**. Pioneira. 2004.

QUADROS, Everton. **Implementando um analisador léxico usando o Flex**. Everton Quadros, 8 set. 2019. Disponível em:
<https://eqdrs.github.io/compilers/2019/09/08/implementando-um-analisador-lexico-usando-o-flex.html>. Acesso em: 19 maio 2026.